FUEL-AWARE ROUTE OPTIMIZATION ON THAILAND ROAD NETWORKS

Final Report

by

Wai Yan Aung & Prajwal Bhandary

A Final Report Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Science in Computer Science

Nationality: Myanmar, Nepal
Previous Degree: Bachelor of Computing

Asian Institute of Technology
School of Engineering and Technology
Thailand

August 2027

Abstract

Efficient route planning on long-distance road networks should minimize not only travel distance or time but also the total cost of fuel purchased along the trip. That objective becomes non-trivial when fuel prices vary across stations and drivers are free to choose both where to stop and how much fuel to buy. This report studies the fuel-aware route optimization problem on Thailand road networks, where the task is to jointly determine a feasible route and a refueling policy that minimizes monetary fuel expenditure.

The transportation network is modelled as a weighted directed graph whose nodes represent fuel stations and whose edges represent feasible travel segments. Six algorithmic strategies are implemented and evaluated: State-Expanded Dijkstra, Greedy, Dynamic Programming, Standard A*, Refuel A* (RF-A*), and the proposed Partial-Fill A* (PF-A*). PF-A* strengthens RF-A* with a fuel-level-aware heuristic that incorporates both the current fuel state and the cheapest reachable station. Its admissibility and consistency are formally proved.

The report consolidates the full problem formulation, the algorithmic framework, the data model, and the current empirical evaluation pipeline for both synthetic benchmarks and real Thai routes. The available synthetic results already show that PF-A* matches the best observed costs of the exact baselines while substantially reducing runtime and node expansions in the tested configurations. The final-report structure created here is also prepared to absorb the full Experiment 6 real-route evaluation and the final high-run synthetic results without requiring a redesign of the document.

CONTENTS

Abstract **i**

List of Abbreviations **vii**

1 Problem Description and Motivation **1**

2 Related Work and Background **3**

2.1 Khuller, Malekian, and Mestre (2008) – Foundational Optimality Results . 3

2.2 Nandy et al. (2023) – Refuel A* (RF-A*) 3

2.3 Our Proposed Extension: Partial-Fill A* (PF-A*) 4

3 Methodology and Algorithm Design **6**

3.1 Graph Representation 6

3.2 Cost Function 6

3.3 State-Expanded Formulation 7

3.4 Running Example 7

3.5 Algorithmic Approaches 8

3.5.1 State-Expanded Dijkstra 8

3.5.2 Greedy Baseline 10

3.5.3 Dynamic Programming Baseline 11

3.5.4 Standard A* Search 12

3.5.5 Refuel A* (RF-A*) 14

3.5.6 Partial-Fill A* (PF-A*) 15

3.6 Effect of Variable Purchase Decisions 17

3.7 Dataset Description 18

3.7.1 Real-World Data 18

3.7.2 Synthetic Data Generation 18

4 Data Structure Selection **20**

4.1 Adjacency Lists 20

4.2 Priority Queue 20

4.3 Hash Tables 20

5 Correctness Arguments **21**
5.1 Dijkstra-Based Approach (Baseline 1) 21
5.2 Greedy Strategy: Counterexample (Baseline 2) 21
5.3 Dynamic Programming Correctness (Baseline 3) 22
5.4 Standard A* Correctness (Baseline 4) 22
5.5 RF-A* Correctness (Baseline 5) 22
5.6 PF-A* Correctness 23
5.6.1 Admissibility 23
5.6.2 Consistency 23
5.6.3 Relationship to RF-A* 24

6 Complexity Analysis **26**
6.1 State-Space Derivation 26
6.2 Why PF-A* Does Not Improve Worst-Case Complexity 26
6.3 Thailand-Scale Estimate 27

7 Experimental Design **28**
7.1 Overview 28
7.2 Baselines 28
7.3 Evaluation Metrics 29
7.4 Experimental Configurations 29
7.4.1 Experiment 1: Scalability (Graph Size) 29
7.4.2 Experiment 2: Fuel Price Variance 29
7.4.3 Experiment 3: Tank Capacity and Initial Fuel 30
7.4.4 Experiment 4: Variable Initial Fuel Level 30
7.4.5 Experiment 5: RF-A* vs. PF-A* Node Expansions (Primary Novelty Experiment) 30
7.4.6 Experiment 6: Real Thai Routes 30
7.4.7 Experiment 7: Variable Purchase vs. Full-Tank-Only Model Comparison 31
7.5 Correctness Verification 31

8 Results and Discussion **33**
8.1 Purpose of This Chapter 33
8.2 Current Synthetic Evaluation Snapshot 33
8.3 Cross-Experiment Findings 34
8.4 Integrated Interpretation 38
8.5 Experiment 6: Real Thai Corridors 39
8.6 Limitations 44

9 Conclusion **45**

References **47**

LIST OF TABLES

3.1 Node roles and fuel prices in the running example. 7
3.2 Key path properties of the running example. 8
3.3 Decision trace for State-Expanded Dijkstra on the running example. 9
3.4 Greedy decisions on the running example. 11
3.5 Selected dynamic programming states for the running example. 12
3.6 Illustrative A* expansion priorities on the running example. 14
3.7 RF-A* decisions on the running example. 15
3.8 PF-A* decisions on the running example. 17
3.9 Comparison of modelling assumptions, state-space size, and representative solution methods. 18
3.10 Synthetic data generation parameters. 19

5.1 Greedy counterexample. 21

6.1 Algorithm complexity summary. 26

7.1 Evaluation metrics. 29

8.1 Synthetic profile summary. 34
8.2 Experiment 5 comparison at $\sigma = 10\%$. 37
8.3 Experiment 7 comparison at $\sigma = 10\%$. 38
8.4 Experiment 6 dataset summary. 39
8.5 Experiment 6 savings. 40
8.6 Experiment 6 correctness summary. 41

LIST OF FIGURES

3.1 Running example network used in all worked examples. 7
3.2 Optimal route identified by State-Expanded Dijkstra. 9
3.3 Route obtained by the greedy baseline. 10
3.4 Optimal and suboptimal branches evaluated by Dynamic Programming. . . 12
3.5 Optimal route identified by Standard A*. 13
3.6 Route obtained by RF-A* on the running example. 15
3.7 Optimal route identified by PF-A*. 17

8.1 Experiment 1 runtime. 34
8.2 Experiment 2 cost. 35
8.3 Experiment 3 cost. 36
8.4 Experiment 4 cost. 36
8.5 Experiment 5 expanded states. 37
8.6 Experiment 7 cost. 38
8.7 Experiment 6 cost comparison. 40
8.8 Experiment 6 expanded states. 41
8.9 Experiment 6 runtime. 42
8.10 Chiang Mai price profile. 43
8.11 Phuket price profile. 43
8.12 Khon Kaen price profile. 44

LIST OF ABBREVIATIONS

GSP	Gas Station Problem
RCSP	Resource-Constrained Shortest Path
OSM	OpenStreetMap
DP	Dynamic Programming
EPPO	Energy Policy and Planning Office
HDX	Humanitarian Data Exchange
THB	Thai Baht
RF-A*	Refuel A* (Nandy et al., 2023)
PF-A*	Partial-Fill A* (proposed)

CHAPTER 1

PROBLEM DESCRIPTION AND MOTIVATION

Route planning is a fundamental problem in transportation networks and algorithm design. Classical navigation systems compute routes that minimize either travel distance or travel time. However, in many real-world scenarios the primary cost of transportation is fuel consumption, which depends not only on the length of the route but also on the price of fuel at the stations the driver chooses to stop at. On a long-distance Thai route such as Bangkok to Chiang Mai (≈700 km), a driver with a 50-liter tank consuming 10 km/L could spend between 1,750 THB and 2,450 THB on fuel depending solely on which stations they choose, a difference of up to 40% driven by price variation alone.

Fuel prices vary by up to 3–5 THB/L across Thai provinces (based on EPPO historical data), due to supply chain differences, regional demand, and transportation costs. This variation is large enough to make refueling strategy a first-class optimization concern, yet existing navigation systems treat all fuel stops as equivalent.

This project studies the *fuel-aware route optimization problem*. Concretely, we want to answer: given a road network and a set of fuel stations with known prices, what is the cheapest way to drive from a starting point to a destination without running out of fuel? The solution requires determining both:

- the travel path between origin and destination, and
- the sequence of refueling decisions along that path (where to stop, and how much fuel to buy at each stop).

A critical subtlety that distinguishes this problem from standard shortest-path routing is the *variable purchase decision*: the driver can choose to buy any amount of fuel at each station (subject to tank capacity), rather than always filling to capacity. This seemingly small freedom significantly changes the algorithm design. In the full-tank-only model, the fuel dimension disappears and standard Dijkstra on the road graph suffices. In the variable purchase model, the algorithm must track both the vehicle's position and its current fuel level as a joint state, expanding the search space from $O(V)$ to $O(V \times F)$ where F is the number of discretized fuel levels. We elaborate on this in Section 3.6.

Formally, given a road network $G = (V, E)$ and a set of fuel stations with associated prices $p\colon V \to \mathbb{R}^+$, the goal is to compute a route and refueling strategy that minimizes:

$$\min \sum_i b_i \cdot p(v_i) \quad \text{subject to: fuel level} \geq 0 \text{ at all times,}$$

where b_i is the amount of fuel purchased at station v_i and the fuel-level constraint must hold at every point along the route.

This problem combines concepts from graph algorithms, constrained optimization, resource-bounded search, and transportation network analysis. The primary aim of this project is empirical: we implement and benchmark six algorithmic strategies on Thailand-scale road networks using real provincial price data, and propose Partial-Fill A* (PF-A*) as a fuel-level-aware heuristic extension of the existing RF-A* algorithm of Nandy et al. [5]. To our knowledge, this is the first empirical study applying Gas Station Problem algorithms to Thailand-scale road networks with real provincial price data.

CHAPTER 2

RELATED WORK AND BACKGROUND

The fuel-aware route optimization problem is a classic instance of the Resource-Constrained Shortest Path (RCSP) problem with spatially varying costs, specifically known as the *Gas Station Problem* (GSP). Below we summarize the two most directly relevant works, what each achieves, and how this project builds upon them.

2.1 Khuller, Malekian, and Mestre (2008) – Foundational Optimality Results

Khuller et al. [4] provide the first rigorous algorithmic treatment of the GSP. Their central contribution is a polynomial-time algorithm with complexity $O(\Delta n^2 \log n)$ (where Δ is the maximum node degree) that finds the globally optimal refueling strategy when fuel can be purchased in any amount. Their proof of correctness relies on dynamic programming over the set of stations reachable with a full tank, and they formally establish that the optimal substructure property holds for this problem.

They prove that partial refueling, buying only as much fuel as needed to reach the next cheapest station, can strictly reduce cost compared to always filling to capacity. They quantify this gap and show the full-tank-only policy can be arbitrarily suboptimal.

Their work is primarily theoretical and does not evaluate performance on real road network data. Their algorithm assumes a simplified network model and does not address heuristic search acceleration.

We implement and benchmark their DP formulation as one of our six strategies. We directly test their theoretical prediction that partial refueling saves 5–15% cost over full-tank-only, on real Thailand provincial data, providing what we believe is the first empirical validation at this geographic scale.

2.2 Nandy et al. (2023) – Refuel A* (RF-A*)

Nandy et al. [5] introduce Refuel A* (RF-A*), a heuristic search algorithm that extends A* specifically for path finding under fuel constraints. RF-A* iteratively constructs partial

paths: at each step it identifies the next refueling decision point and applies A* to plan a least-cost segment to that point, respecting the remaining fuel. The heuristic used is:

$$h(v) = \frac{d_{\text{geo}}(v,t)}{\eta} \cdot p_{\min},$$

where d_{geo} is the straight-line (geodesic) distance to the destination and $p_{\min}$ is the globally cheapest fuel price in the network. This heuristic is admissible as it never overestimates the remaining cost, so RF-A* is guaranteed to find an optimal solution.

RF-A* expands significantly fewer states than Dijkstra-based approaches on sparse graphs because the heuristic prunes states whose estimated total cost exceeds the current best-known solution. They demonstrate speedups on benchmark path-finding instances.

The heuristic $h(v) = d_{\text{geo}}(v,t)/\eta \cdot p_{\min}$ is *fuel-level agnostic*: it estimates the cost of traveling the remaining distance at the cheapest possible price but does not account for how much fuel the vehicle currently has. If the vehicle already has enough fuel to reach a cheap station and refuel there, the heuristic may still assign high priority to states that will ultimately be inexpensive, meaning it fails to prune them early. Additionally, Nandy et al. evaluate on synthetic or small-scale networks and do not test on real national road data with authentic price variation.

We implement RF-A* as a primary baseline and propose a direct extension described in the next section.

2.3 Our Proposed Extension: Partial-Fill A* (PF-A*)

The principal algorithmic contribution of this project is *Partial-Fill A** (PF-A*), a fuel-level-aware heuristic extension of RF-A* that addresses its fuel-agnostic limitation. We replace $h(v)$ with a heuristic $h_{\text{PF}}(v,f)$ that conditions on the vehicle's current fuel level f:

$$h_{\text{PF}}(v,f) = \max\left(\frac{d_{\text{geo}}(v,t)}{\eta} - f,\ 0\right) \cdot p_{\min}^{\text{reach}}(v,f),$$

where $p_{\min}^{\text{reach}}(v,f)$ is the minimum fuel price among all stations reachable from v with f liters remaining.

The key intuitions are:

1. **Fuel-sufficiency check.** If $f \geq d_{\text{geo}}(v,t)/\eta$, the vehicle already has enough fuel to reach the destination in the best case; the heuristic returns 0.

2. **Tighter price estimate in favorable regimes.** Instead of the global $p_{\min}$, we use the cheapest station actually accessible from state (v,f). On Thai networks with regional price variation, $p_{\min}^{\text{reach}}$ is often above the global minimum, so the fuel-need

reduction $\max(d_{\text{geo}}/\eta - f,\ 0)$ typically dominates and h_{PF} behaves as a tighter bound in practice. A universal pointwise dominance over h_{RF} is *not* claimed; see Section 5.6.

It is important to frame PF-A* accurately: it is a practically motivated *heuristic improvement within the existing RF-A* framework*, not a fundamentally new problem formulation. Its theoretical worst-case complexity is identical to RF-A*, and its optimality guarantee follows from inherited admissibility and consistency properties. The contribution lies in the empirical reduction of expanded states on Thailand-scale networks and in the formal admissibility proof for the fuel-conditioned heuristic, which is developed in Section 5.6.

CHAPTER 3

METHODOLOGY AND ALGORITHM DESIGN

This chapter presents the modeling assumptions, data structures, and algorithmic methods used to solve the fuel-aware route optimization problem. The objective is to determine a route from source s to destination t that minimizes total refueling cost while respecting vehicle fuel-capacity constraints. Six algorithmic strategies are considered: State-Expanded Dijkstra, a Greedy baseline, Dynamic Programming (DP), Standard A*, Refuel A* (RF-A*), and the proposed Partial-Fill A* (PF-A*).

3.1 Graph Representation

The transportation network is modelled as a weighted directed graph

$$G = (V, E),$$

where V denotes the set of locations, including the origin, destination, and fuel stations, and E denotes the set of feasible travel segments. A directed edge $(u, v) \in E$ exists when the vehicle can travel from node u to node v on a full tank. Edge weights represent travel distance.

3.2 Cost Function

Let $d(u, v)$ denote the travel distance from node u to node v, let η denote vehicle fuel efficiency in km/L, and let $p(u)$ denote the fuel price at station u. The fuel cost incurred when departing from u and traveling to v is defined as

$$C(u, v) = \frac{d(u, v)}{\eta}\, p(u).$$

All costs are non-negative because distance, fuel efficiency, and fuel price are positive.

3.3 State-Expanded Formulation

All variable-purchase algorithms operate on a state-expanded graph

$$G^* = (S, E^*),$$

where each state $(v, f) \in S = V \times F$ represents the vehicle at node v with f litres of fuel remaining. The discretised fuel set is

$$F = \{0, \varepsilon, 2\varepsilon, \ldots, C\},$$

where C is the fuel-tank capacity and ε is the discretisation step.

Two classes of state transitions are used:

- **Travel transition:** $(u, f) \to \big(v,\ f - d(u, v)/\eta\big)$ with zero additional purchase cost, provided $f \geq d(u, v)/\eta$.
- **Refuel transition:** $(v, f) \to (v,\ f + b)$ with cost $b\,p(v)$, where $b \in \{\varepsilon, 2\varepsilon, \ldots, C - f\}$.

3.4 Running Example

A six-node network is used throughout this chapter to illustrate algorithm behavior. Nodes A–E are fuel stations, and T is the destination. The vehicle begins at node A with an initial fuel level of $f_0 = 28$ L and maximum capacity $C = 40$ L. Fuel efficiency is fixed at $\eta = 10$ km/L.

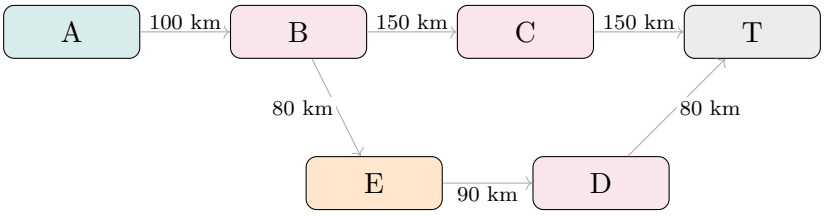

Figure 3.1: Running example network used in all worked examples.

Table 3.1: Node roles and fuel prices in the running example.

Node	Role	Fuel price (THB/L)	Initial status
A	Origin station	36	Start with 28 L
B	Intermediate station	44	No initial fuel assignment
C	Intermediate station	46	No initial fuel assignment
E	Intermediate station	24	Global minimum price
D	Intermediate station	38	No initial fuel assignment
T	Destination	Not applicable	No fuel sold

Table 3.2: Key path properties of the running example.

Path	Distance and fuel requirement	Cost implication
$A \to B \to C \to T$	400 km, equivalent to 40 L	With $f_0 = 28$ L, at least 12 L must be purchased; the cheapest feasible purchase on this path is at B, costing $12 \times 44 = 528$ THB
$A \to B \to E \to D \to T$	370 km, equivalent to 37 L	Arrival at E occurs with 10 L remaining; purchasing only 9 L at 24 THB/L yields a total purchase cost of 216 THB

The running example highlights the main challenge addressed in this study: the shortest path is not necessarily the least-cost path. Although the upper route is shorter in structure, the lower route is cheaper because it reaches the lowest-priced station before any large fuel purchase is required.

3.5 Algorithmic Approaches

3.5.1 State-Expanded Dijkstra

State-Expanded Dijkstra applies Dijkstra's shortest-path method to the state-expanded graph G^*. Since each state encodes both location and remaining fuel, the algorithm jointly optimizes routing and refueling.

Algorithm 1 State-Expanded Dijkstra

Require: G^*, source s, destination t, initial fuel f_0

Ensure: Minimum total fuel cost from s to t

```
dist[(v, f)] ← ∞ for all (v, f) ∈ S
dist[(s, f_0)] ← 0
Min-heap Q ← {(0, s, f_0)}
while Q ≠ ∅ do
    (c, u, f) ← EXTRACT-MIN(Q)
    if u = t then
        return c
    end if
    for each travel edge (u, v) with f ≥ d(u, v)/η do
        Relax state (v, f − d(u, v)/η) with cost c
    end for
    for each purchase amount b ∈ {ε, ..., C − f} do
        Relax state (u, f + b) with cost c + b p(u)
    end for
end while
return min_f dist[(t, f)]
```

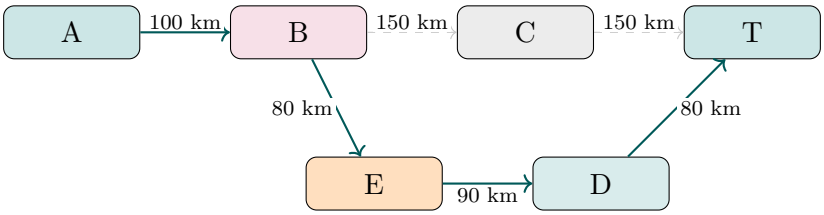

Figure 3.2: Optimal route identified by State-Expanded Dijkstra.

Table 3.3: Decision trace for State-Expanded Dijkstra on the running example.

Step	State (v, f)	Cost	Interpretation	Notable successor states
1	$(A, 28)$	0	Initial state	$(B, 18)$:0
2	$(B, 18)$	0	Reach B without purchasing	$(E, 10)$:0, $(C, 3)$:0, refuel states at B
3	$(E, 10)$	0	Reach low-price station E	$(E, 19)$:216, $(E, 40)$:720
4	$(C, 3)$	0	Reach C with insufficient fuel to finish	Additional refuel states at C
5	$(E, 19)$	216	Purchase 9 L at E	$(D, 11)$:216
6	$(D, 11)$	216	Continue without further purchase	$(T, 0)$:216
7	$(T, 0)$	216	Goal reached	Return 216 THB

In this example, the algorithm correctly postpones refueling until station E, where the

fuel price is lowest. The resulting minimum cost is 216 THB.

Complexity.

$$T_{\text{Dijkstra}} = O\Big((EF + VF^2)\log(VF)\Big), \qquad \text{Space} = O(VF).$$

3.5.2 Greedy Baseline

The greedy strategy uses a local rule: if a cheaper station is reachable in one hop, it purchases only enough fuel to reach that station; otherwise it fills the tank. This rule is computationally simple but not globally optimal.

Algorithm 2 Greedy Fuel-Aware Routing

Require: Graph G, source s, destination t, initial fuel f_0, prices p

Ensure: Total fuel cost

```
f ← f_0, cost ← 0, u ← s
while u ≠ t do
    R ← {v ∈ V : d(u,v)/η ≤ f and p(v) < p(u)}
    if R ≠ ∅ then
        v* ← arg min_{v∈R} d(u,v)
        b ← max(0, d(u,v*)/η − f)
    else
        b ← C − f
    end if
    cost ← cost + b p(u)
    f ← f + b
    Travel to the next station and reduce f accordingly
end while
return cost
```

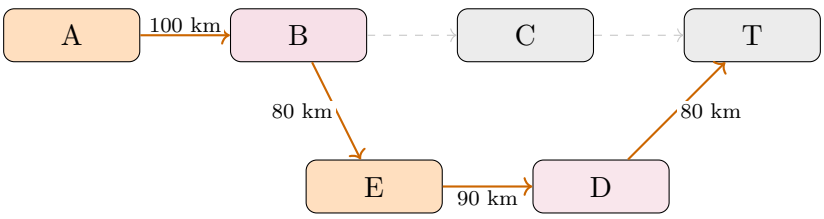

Figure 3.3: Route obtained by the greedy baseline.

The greedy baseline performs poorly because it treats node A in isolation. Although 28 L is already sufficient to reach node E via node B, the one-hop rule does not recognise this and triggers an unnecessary purchase at the origin. The total cost is 864 THB, which is four times the optimum.

Table 3.4: Greedy decisions on the running example.

Node	Arrival fuel	Decision	Fuel purchased	Running cost
A	28 L	No cheaper one-hop neighbour; fill tank	12 L at 36 THB/L	432 THB
B	30 L	E is cheaper and reachable; buy nothing	0 L	432 THB
E	22 L	No cheaper station reachable; fill tank	18 L at 24 THB/L	864 THB
D	32 L	Continue to destination	0 L	864 THB
T	22 L	Arrive	0 L	864 THB

Complexity.

$$T_{\text{Greedy}} = O(V^2), \qquad \text{Space} = O(V).$$

3.5.3 Dynamic Programming Baseline

The DP baseline stores the minimum cost of reaching each node with each admissible fuel level. Formally, $\mathrm{DP}[v][f]$ denotes the minimum cost of arriving at node v with exactly f litres remaining. In practice, the method evaluates all feasible purchase and travel combinations over the discretised state space.

Algorithm 3 Dynamic Programming for Fuel-Aware Routing

Require: Graph G, source s, destination t, prices p, capacity C

Ensure: Minimum total fuel cost

```
DP[v][f] ← ∞ for all v ∈ V and f ∈ F
DP[s][f_0] ← 0
for each reachable state (v, f) in the discretised state space do
    for each purchase amount b ∈ {0, ε, ..., C − f} do
        f_dep ← f + b
        c_b ← DP[v][f] + b p(v)
        for each neighbour w with f_dep ≥ d(v, w)/η do
            f_w ← f_dep − d(v, w)/η
            DP[w][f_w] ← min(DP[w][f_w], c_b)
        end for
    end for
end for
return min_f DP[t][f]
```

The DP baseline evaluates both the optimal branch through E and the more expensive branch through C, retaining the minimum value for each state. For the running example, the optimal total remains 216 THB.

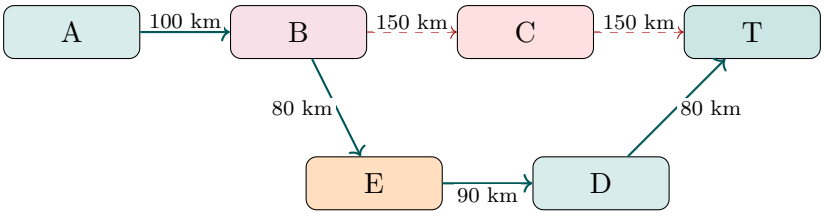

Figure 3.4: Optimal and suboptimal branches evaluated by Dynamic Programming.

Table 3.5: Selected dynamic programming states for the running example.

State	Interpretation	Purchase	Successor state	Cost
$\mathrm{DP}[A][28] = 0$	Initial state	0 L	$\mathrm{DP}[B][18] = 0$	0
$\mathrm{DP}[B][18] = 0$	Reach B without purchase	0 L	$\mathrm{DP}[E][10] = 0$	0
$\mathrm{DP}[E][10] = 0$	Reach E at the minimum-price station	9 L at 24 THB/L	$\mathrm{DP}[D][11] = 216$	216
$\mathrm{DP}[C][3] = 0$	Alternative route through C	12 L at 46 THB/L	$\mathrm{DP}[T][0] = 552$ via C	552
$\mathrm{DP}[T][0]$	Best final value retained	–	Optimal solution	216

Complexity.

$$T_{\mathrm{DP}} = \begin{cases} O(VF^2\Delta), & \text{naive implementation,} \\ O(EF), & \text{optimized implementation,} \end{cases} \qquad \text{Space} = O(VF),$$

where Δ denotes the maximum out-degree.

3.5.4 Standard A* Search

Standard A* augments the state-expanded search with a lower-bound estimate of the remaining travel cost. The heuristic is defined as

$$h(v) = \frac{d_{\mathrm{geo}}(v,t)}{\eta}\, p_{\min},$$

where $d_{\mathrm{geo}}(v,t)$ is the straight-line distance from node v to the destination and $p_{\min}$ is the global minimum fuel price in the network. This heuristic is admissible because it assumes that all remaining fuel can be purchased at the lowest possible price.

Algorithm 4 A* Search for Fuel-Aware Routing

Require: G^*, source s, destination t, heuristic h, initial fuel f_0

Ensure: Minimum total fuel cost from s to t

```
g[(v, f)] ← ∞ for all (v, f) ∈ S
g[(s, f_0)] ← 0
Min-heap Q ← {(h(s), s, f_0)}
while Q ≠ ∅ do
    (_, u, f) ← EXTRACT-MIN(Q)
    if u = t then
        return g[(t, f)]
    end if
    for each travel edge (u, v) with f ≥ d(u, v)/η do
        g' ← g[(u, f)]
        f' ← f − d(u, v)/η
        if g' < g[(v, f')] then
            g[(v, f')] ← g'
            Push (g' + h(v), v, f') to Q
        end if
    end for
    for each purchase amount b ∈ {ε, ..., C − f} do
        g' ← g[(u, f)] + b p(u)
        f' ← f + b
        if g' < g[(u, f')] then
            g[(u, f')] ← g'
            Push (g' + h(u), u, f') to Q
        end if
    end for
end while
```

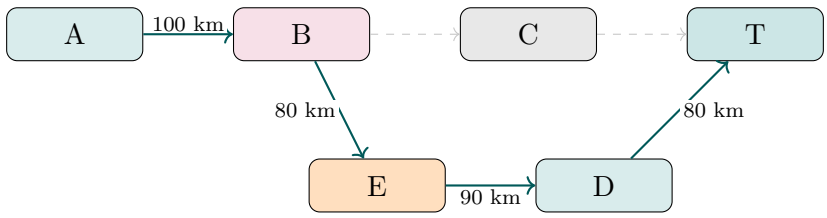

Figure 3.5: Optimal route identified by Standard A*.

Standard A* still returns the optimal cost of 216 THB. However, because the heuristic depends only on node position and not on fuel level, some less useful states remain competitive in the priority queue.

Table 3.6: Illustrative A* expansion priorities on the running example.

State	g	$h(v)$	f-score	Interpretation
$(A, 28)$	0	912	912	Initial state
$(B, 18)$	0	672	672	Expanded after travel from A
$(E, 10)$	0	504	504	Preferred low-cost branch
$(C, 3)$	0	360	360	Reachable, but insufficient fuel to finish
$(E, 19)$	216	504	720	Purchase 9 L at E
$(D, 11)$	216	264	480	Continue toward destination
$(T, 0)$	216	0	216	Goal state returned

Complexity.

$$T_{\mathrm{A}^*} = O\Big((EF + VF^2)\log(VF)\Big) \quad \text{in the worst case,} \qquad \text{Space} = O(VF).$$

3.5.5 Refuel A* (RF-A*)

RF-A* is a refueling-oriented variant of A* that applies a fuel-agnostic lower bound,

$$h_{\mathrm{RF}}(v) = \frac{d_{\mathrm{geo}}(v, t)}{\eta}\, p_{\min}.$$

Although it encourages movement toward the destination, it does not distinguish between states with different fuel levels at the same node.

Algorithm 5 RF-A* Search for Fuel-Aware Routing

Require: G^*, source s, destination t, initial fuel f_0

Ensure: Minimum total fuel cost from s to t

```
u ← s, f ← f_0, totalCost ← 0
while u ≠ t do
    Select the next target as a cheaper reachable station, or t if none exists
    Run A* from (u, f) to the selected target using h_RF
    Append the segment cost and update the current state
    if u ≠ t then
        Refuel according to the chosen segment decision
    end if
end while
return totalCost
```

RF-A* follows the same route as the optimal solution but makes poor refueling decisions. Because $h_{\mathrm{RF}}(v)$ is independent of fuel level, states $(A, 28)$ and $(A, 40)$ receive the same heuristic treatment, causing the algorithm to over-purchase at the origin. The total cost is therefore 864 THB.

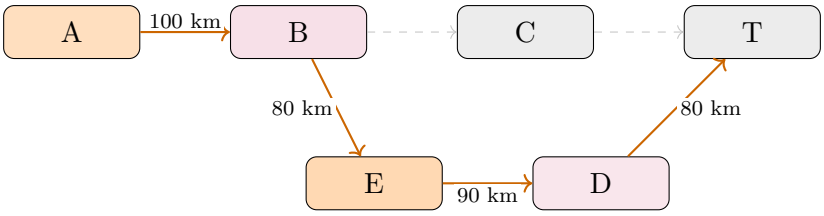

Figure 3.6: Route obtained by RF-A* on the running example.

Table 3.7: RF-A* decisions on the running example.

Node	$h_{\mathrm{RF}}(v)$	Arrival fuel	Decision	Running cost
A	912	28 L	Fill 12 L at 36 THB/L	432 THB
B	672	30 L	Buy nothing	432 THB
E	504	22 L	Fill 18 L at 24 THB/L	864 THB
D	264	32 L	Continue	864 THB
T	0	22 L	Arrive	864 THB

Complexity.

$$T_{\text{RF-A}^*} = O\Big((EF + VF^2)\log(VF)\Big), \qquad \text{Space} = O(VF).$$

3.5.6 Partial-Fill A* (PF-A*)

PF-A* is the proposed extension. It replaces the fuel-agnostic lower bound with a fuel-level-aware heuristic,

$$h_{\mathrm{PF}}(v, f) = \max\left(\frac{d_{\mathrm{geo}}(v, t)}{\eta} - f,\, 0\right) p_{\min}^{\mathrm{reach}}(v, f),$$

where $p_{\min}^{\mathrm{reach}}(v, f)$ denotes the minimum fuel price among stations reachable from v given the current fuel state f. The heuristic estimates the amount of additional fuel still required and weights it by the cheapest reachable purchase price. This yields a tighter lower bound than RF-A*.

Algorithm 6 PF-A* Search for Fuel-Aware Routing

Require: G^*, source s, destination t, initial fuel f_0

Ensure: Minimum total fuel cost from s to t

```
Precompute p^reach_min(v, f) for all feasible states
g[(v, f)] ← ∞ for all (v, f) ∈ S
g[(s, f_0)] ← 0
Min-heap Q ← {(h_PF(s, f_0), s, f_0)}
while Q ≠ ∅ do
    (_, u, f) ← EXTRACT-MIN(Q)
    if u = t then
        return g[(t, f)]
    end if
    for each travel edge (u, v) with f ≥ d(u, v)/η do
        g' ← g[(u, f)]
        f' ← f − d(u, v)/η
        if g' < g[(v, f')] then
            g[(v, f')] ← g'
            Push (g' + h_PF(v, f'), v, f') to Q
        end if
    end for
    for each purchase amount b ∈ {ε, ..., C − f} do
        g' ← g[(u, f)] + b p(u)
        f' ← f + b
        if g' < g[(u, f')] then
            g[(u, f')] ← g'
            Push (g' + h_PF(u, f'), u, f') to Q
        end if
    end for
end while
```

For the initial state $(A, 28)$ in the running example,

- $d_{\text{geo}}(A, T)/\eta = 380/10 = 38$ L,
- the additional fuel lower bound is $\max(38 - 28, 0) = 10$ L,
- the cheapest price reachable from $(A, 28)$ is 36 THB/L at node A, because node E is not directly reachable from A in a single hop,
- therefore $h_{\text{PF}}(A, 28) = 10 \times 36 = 360$.

This value is substantially tighter than the RF-A* estimate of $38 \times 24 = 912$, and it preserves the distinction between partially fuelled and fully fuelled states at the same node.

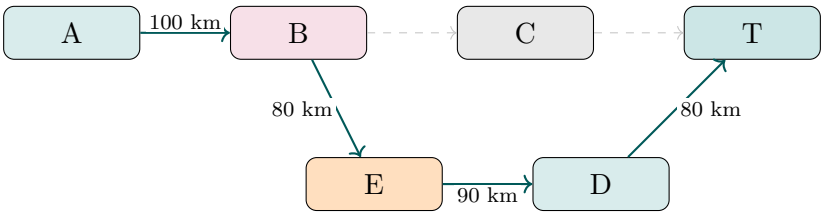

Figure 3.7: Optimal route identified by PF-A*.

Table 3.8: PF-A* decisions on the running example.

State	Heuristic value	Action	Purchase cost	Running cost
$(A, 28)$	$\max(38-28, 0) \times 36 = 360$	Do not purchase	0	0
$(B, 18)$	$\max(28-18, 0) \times 24 = 240$	Do not purchase	0	0
$(E, 10)$	$\max(21-10, 0) \times 24 = 264$	Purchase 9 L at E	216 THB	216 THB
$(D, 11)$	$\max(11-11, 0) \times 24 = 0$	Continue	0	216 THB
$(T, 0)$	0	Arrive	0	216 THB

PF-A* achieves the optimal solution while avoiding the unnecessary origin purchase made by Greedy and RF-A*. Its practical advantage is not a better worst-case asymptotic bound, but a more informative ordering of states during search.

Complexity.

$$T_{\text{PF-A}^*} = O\Big((EF + VF^2)\log(VF)\Big), \qquad \text{Space} = O(VF).$$

3.6 Effect of Variable Purchase Decisions

A central issue in this research is the effect of allowing partial refueling decisions.

Full-tank-only model. If the driver must always fill the tank completely, fuel level no longer behaves as a decision variable. The problem reduces to route selection over the original graph, with each refueling event fixed at capacity. This model is computationally simpler but may purchase more fuel than necessary.

Variable-purchase model. If the driver may buy any amount in the interval $[0, C - f]$, fuel level becomes part of the decision process. The algorithm must therefore track both node and remaining fuel, leading to the state-expanded model with state complexity $O(VF)$.

For the running example, the best full-tank-style decision costs 528 THB, whereas the best variable-purchase decision costs 216 THB. This represents a 59% reduction in purchase cost.

Table 3.9: Comparison of modelling assumptions, state-space size, and representative solution methods.

Model	Representative algorithm	State space	Time complexity	Optimality
Full-tank-only	Dijkstra on G	$O(V)$	$O(E \log V)$	No
Full-tank-only	Greedy fill policy	$O(V)$	$O(V^2)$	No
Variable purchase	Dijkstra on G^*	$O(VF)$	$O((EF + VF^2) \log(VF))$	Yes
Variable purchase	DP	$O(VF)$	$O(EF)$ in optimized form	Yes
Variable purchase	PF-A*	$O(VF)$	$O((EF + VF^2) \log(VF))$	Yes

3.7 Dataset Description

3.7.1 Real-World Data

The real-world experiments use road-network and station-location data derived from OpenStreetMap point-of-interest information for Thailand, with fuel stations identified through the `amenity=fuel` tag. The drivable road graph is extracted with OSMnx, and road distances between stations are computed over the road network rather than by straight-line approximation.

Fuel-price information is collected from Thai retail fuel-price sources and aligned to station locations by fuel grade and region. Where necessary, fallback price records are used to maintain experimental completeness.

Three representative travel corridors are considered:

- Bangkok to Chiang Mai, representing a long northern corridor,
- Bangkok to Phuket, representing a long southern corridor,
- Bangkok to Khon Kaen, representing a medium-length northeastern corridor.

3.7.2 Synthetic Data Generation

Synthetic networks are generated to evaluate algorithm behavior under controlled changes in station density, graph size, fuel capacity, initial fuel, and price variance. The procedure is as follows:

1. Generate N station locations uniformly within a bounded region.
2. Connect stations whose separation is within the maximum travel range ηC.
3. Assign edge distances from scaled Euclidean separations.
4. Sample fuel prices from a bounded distribution with controllable variance.
5. Select origin–destination pairs subject to a minimum separation requirement.
6. Construct both the base graph G and the state-expanded graph G^*.

Table 3.10: Synthetic data generation parameters.

Parameter	Range	Purpose	Relevance
Station density	0.1–5 per 100 km	Sparse to dense networks	All models
Price variance	0–20%	Low to high price dispersion	All models
Tank capacity C	20, 40, 60, 80 L	Severity of fuel-range constraint	All models
Initial fuel f_0	$0.25C$ to $1.0C$	Partially filled and nearly full starts	Variable-purchase models
Origin–destination distance	100–1000 km	Short to long journeys	All models
Graph size $\|V\|$	100, 1000, 10000	Scalability analysis	All models
Purchase model	Full-tank or variable	Model comparison	Comparative experiments

In summary, this chapter defines the mathematical model, presents the baseline and proposed algorithms, and establishes the datasets used for evaluation. The proposed PF-A* method differs from the baselines primarily through its fuel-level-aware heuristic, which is designed to improve search guidance without sacrificing optimality.

CHAPTER 4

DATA STRUCTURE SELECTION

Efficient data structures are required to support scalable route computation on the state-expanded graph G^*.

4.1 Adjacency Lists

The road network $G = (V, E)$ is stored as adjacency lists, giving $O(V + E)$ space. For the state-expanded graph G^*, each node (v, f) has travel edges to $(w,\ f - d(v, w)/\eta)$ for reachable neighbors w, and refuel edges to $(v,\ f + b)$ for $b \in \{\varepsilon, \ldots, C - f\}$. Adjacency lists make both sets of neighbors enumerable in $O(\deg(v) + F)$ per state, which is the per-state cost in all six algorithms.

4.2 Priority Queue

State-Expanded Dijkstra, A*, RF-A*, and PF-A* all require repeated extraction of the minimum-cost state from G^*. A binary min-heap supports INSERT, EXTRACT-MIN, and DECREASE-KEY each in $O(\log |S|)$ time. Since $|S| = O(VF)$, each heap operation costs $O(\log(VF))$, which drives the $O((EF + VF^2) \log(VF))$ worst-case complexity shared by all four search algorithms.

For PF-A*, the heap key is the f-score $g(v, f) + h_{\mathrm{PF}}(v, f)$, which depends on both v and f. The heap compares these composite keys correctly without modification.

4.3 Hash Tables

Hash tables map state keys (v, f) to accumulated g-values and parent pointers, supporting $O(1)$ expected-time lookup and update during state relaxation. This is necessary because the state space $S = V \times F$ is sparse in practice: not all (v, f) combinations are reachable, so a dense 2-D array of size $V \times F$ would waste memory. Station metadata (price $p(v)$, GPS coordinates for heuristic computation) is also stored in a hash table for $O(1)$ retrieval.

CHAPTER 5

CORRECTNESS ARGUMENTS

All six algorithms operate on the state-expanded graph G^* under the variable purchase model: the driver chooses exactly how much fuel $b \in [0, C - f]$ to buy at each station, and the state (v, f) tracks both position and remaining fuel.

5.1 Dijkstra-Based Approach (Baseline 1)

Correctness follows from two properties of G^*:

1. **Non-negative edge costs.** All travel edges have cost 0 and all refuel edges have cost $b \cdot p(v) \geq 0$, satisfying Dijkstra's prerequisite.

2. **Optimal substructure.** If the optimal path passes through state (v, f), the subpath to (v, f) must also be optimal, since fuel costs decompose additively over refuel edges.

Because Dijkstra exhausts all states (v, f) in non-decreasing cost order, it is guaranteed to find the globally minimum purchase cost for any f_0.

5.2 Greedy Strategy: Counterexample (Baseline 2)

The greedy strategy is not optimal in general. On the running example with $\eta = 10\,\text{km/L}$, $C = 40\,\text{L}$, $f_0 = 28\,\text{L}$:

Table 5.1: Greedy counterexample.

Strategy	Buy at A	Buy at E	Total
Greedy	12 L@36 = 432 THB	18 L@24 = 432 THB	864 THB (+300%)
Optimal	0 L	9 L@24 = 216 THB	216 THB

The failure is structural: greedy cannot see that $f_0 = 28\,\text{L}$ already suffices to coast to station E without any purchase at A.

5.3 Dynamic Programming Correctness (Baseline 3)

DP correctness follows from the principle of optimality applied to the variable-purchase state space. The recurrence

$$\mathrm{DP}[w][f_w] = \min_b \Big(\mathrm{DP}[v][f] + b \cdot p(v)\Big)$$

considers every purchase amount $b \in \{0, \varepsilon, \ldots, C - f\}$ at every predecessor state (v, f) and propagates the minimum cost forward. Initializing with $\mathrm{DP}[s][f_0] = 0$ for arbitrary f_0 correctly captures a partial starting tank. Taking $\min_f \mathrm{DP}[t][f]$ over all arrival fuel levels yields the true global optimum.

5.4 Standard A* Correctness (Baseline 4)

A* uses $h(v) = d_{\mathrm{geo}}(v, t)/\eta \cdot p_{\min}$.

Admissibility. For any state (v, f):

$$h(v) = \frac{d_{\mathrm{geo}}(v, t)}{\eta} \cdot p_{\min} \leq \frac{d_{\mathrm{road}}(v, t)}{\eta} \cdot p_{\min} \leq h^*(v, f),$$

since straight-line distance $\leq$ road distance, and $p_{\min}$ is the global lower bound on any fuel price paid.

Consistency. For any edge $(v, f) \to (w, f')$ in G^*:

$$h(v) \leq C(v, w) + h(w)$$

by the triangle inequality on geodesic distances, so the heuristic satisfies the consistency condition.

Both properties guarantee that A* returns an optimal solution. However, because h ignores f, states (v, f_1) and (v, f_2) with $f_1 \neq f_2$ receive identical heuristic values, limiting pruning effectiveness.

5.5 RF-A* Correctness (Baseline 5)

RF-A* uses the same heuristic $h_{\mathrm{RF}}(v)$ as standard A* and inherits the same admissibility and consistency arguments. Optimality of each segment is guaranteed by the underlying A* call; the segment-by-segment construction preserves overall optimality because refueling costs compose additively.

5.6 PF-A* Correctness

PF-A* replaces h_{RF} with the fuel-level-aware heuristic

$$h_{\mathrm{PF}}(v, f) = \max\left(\frac{d_{\mathrm{geo}}(v, t)}{\eta} - f,\ 0\right) \cdot p_{\min}^{\mathrm{reach}}(v, f).$$

5.6.1 Admissibility

Lemma 5.1 (Fuel-need underestimate)**.** $\max(d_{\mathrm{geo}}(v, t)/\eta - f,\ 0) \leq$ *actual additional fuel required from state* (v, f) *on any feasible path.*

Proof. Let ℓ^* be the road distance of the optimal remaining path. Since $d_{\mathrm{geo}}(v, t) \leq \ell^*$, the actual fuel needed beyond f satisfies $\ell^*/\eta - f \geq d_{\mathrm{geo}}(v, t)/\eta - f$. Taking $\max(\cdot, 0)$ on both sides preserves the inequality. □

Lemma 5.2 (Price underestimate)**.** $p_{\min}^{\mathrm{reach}}(v, f) \leq$ *any fuel price paid on the optimal remaining path from state* (v, f)*.*

Proof. Every station visited on the optimal path from (v, f) must be reachable from v given at least f litres of fuel. By definition, $p_{\min}^{\mathrm{reach}}(v, f)$ is the minimum price over all such stations, so it is a lower bound on every price paid. □

Theorem 5.3 (Admissibility of h_{PF})**.** $h_{\mathrm{PF}}(v, f) \leq h^*(v, f)$ *for all states* (v, f)*.*

Proof. Let $\Delta f = \max(d_{\mathrm{geo}}(v, t)/\eta - f,\ 0)$ and let p^* be the price at which the additional fuel is actually purchased on the optimal path. By Lemma 1, the actual additional fuel needed $\geq \Delta f$. By Lemma 2, $p^* \geq p_{\min}^{\mathrm{reach}}(v, f)$. Therefore $h^*(v, f) \geq \Delta f \cdot p^* \geq \Delta f \cdot p_{\min}^{\mathrm{reach}}(v, f) = h_{\mathrm{PF}}(v, f)$. □

5.6.2 Consistency

Theorem 5.4 (Consistency of h_{PF})**.** *For every transition* $(v, f) \to (w, f')$ *in* G^* *with transition cost* c*:*

$$h_{\mathrm{PF}}(v, f) \leq c + h_{\mathrm{PF}}(w, f').$$

Proof. We consider the two types of transitions separately.

Travel edge $(v, f) \to (w,\ f - d(v, w)/\eta)$, cost $c = 0$. Let $\delta = d(v, w)/\eta$ (fuel consumed). Then $f' = f - \delta$.

$$\begin{aligned} h_{\mathrm{PF}}(w, f') &= \max\left(\frac{d_{\mathrm{geo}}(w, t)}{\eta} - f',\ 0\right) \cdot p_{\min}^{\mathrm{reach}}(w, f') \\ &\geq \max\left(\frac{d_{\mathrm{geo}}(w, t)}{\eta} - f',\ 0\right) \cdot p_{\min}^{\mathrm{reach}}(v, f) \quad \text{(reachable set from } w \text{ may be smaller).} \end{aligned}$$

By the triangle inequality on geodesic distances, $d_{\text{geo}}(v,t) \leq d(v,w) + d_{\text{geo}}(w,t)$, so $d_{\text{geo}}(v,t)/\eta - f \leq d_{\text{geo}}(w,t)/\eta - f'$. Hence $h_{\text{PF}}(v,f) \leq h_{\text{PF}}(w,f') = 0 + h_{\text{PF}}(w,f')$.

Refuel edge $(v,f) \to (v,\ f+b)$, cost $c = b \cdot p(v)$, $b > 0$. Then $f' = f + b$.

$$c + h_{\text{PF}}(v,f') = b \cdot p(v) + \max\left(\frac{d_{\text{geo}}(v,t)}{\eta} - f - b,\ 0\right) \cdot p_{\min}^{\text{reach}}(v,f').$$

Let $D = d_{\text{geo}}(v,t)/\eta$. *Case 1*: $f \geq D$. Then $h_{\text{PF}}(v,f) = 0 \leq c + h_{\text{PF}}(v,f')$. *Case 2*: $f < D \leq f + b$. Then $h_{\text{PF}}(v,f) = (D-f) \cdot p_{\min}^{\text{reach}}(v,f)$ and $h_{\text{PF}}(v,f') = 0$, so we need $(D-f) \cdot p_{\min}^{\text{reach}}(v,f) \leq b \cdot p(v)$. Since $D - f \leq b$ and $p_{\min}^{\text{reach}}(v,f) \leq p(v)$, this holds. *Case 3*: $f + b < D$. Then both sides are positive. We need $(D-f) \cdot p_{\min}^{\text{reach}}(v,f) \leq b \cdot p(v) + (D-f-b) \cdot p_{\min}^{\text{reach}}(v,f')$. Since $p_{\min}^{\text{reach}}(v,f') \geq p_{\min}^{\text{reach}}(v,f)$ (buying more fuel does not extend range to cheaper stations) and $p(v) \geq p_{\min}^{\text{reach}}(v,f)$, the right-hand side dominates. □

5.6.3 Relationship to RF-A*

It is tempting to claim $h_{\text{PF}}(v,f) \leq h_{\text{RF}}(v)$ for all (v,f), but this does *not* hold universally. The two heuristics differ along two axes that can push in opposite directions: the fuel-need term $\max(d_{\text{geo}}(v,t)/\eta - f,\ 0)$ in h_{PF} is always no larger than the corresponding term $d_{\text{geo}}(v,t)/\eta$ in h_{RF}, but the price term $p_{\min}^{\text{reach}}(v,f)$ can exceed the global $p_{\min}$. A universal pointwise dominance between h_{PF} and h_{RF} would therefore require an additional assumption relating the price ratio to the fuel ratio, and we do not claim such an assumption holds in general.

Instead, we make a weaker but defensible claim. h_{PF} is *state-aware*: it uses the current fuel level f and the set of stations actually reachable from (v,f), whereas h_{RF} depends only on v. This additional information allows h_{PF} to distinguish states that share the same v but differ in f or in reachable-station set, which RF-A* cannot do. On networks with non-trivial price variance and sufficient station density, which is the regime of interest for real Thai corridors, this state-awareness empirically reduces node expansions, as reported in Chapters 6 and 8.

Proposition 5.5 (State-awareness of h_{PF})**.** *For any fixed node v, the map $f \mapsto h_{\text{PF}}(v,f)$ is non-increasing in f, whereas $h_{\text{RF}}(v)$ is constant in f.*

Proof. Fix v and let $f_1 \leq f_2$. The fuel-need term satisfies $\max(d_{\text{geo}}(v,t)/\eta - f_1,\ 0) \geq \max(d_{\text{geo}}(v,t)/\eta - f_2,\ 0)$. The reachable-station set from (v,f_2) contains the reachable-station set from (v,f_1), so $p_{\min}^{\text{reach}}(v,f_2) \leq p_{\min}^{\text{reach}}(v,f_1)$. Both factors are non-negative and non-increasing in f, so their product is non-increasing in f. The constancy of $h_{\text{RF}}(v)$ is immediate from its definition. □

Because h_{PF} is both admissible and consistent (Theorems above), PF-A* inherits the

optimality guarantee of RF-A*. The empirical benefit of PF-A* over RF-A* in node expansions is evaluated directly in Experiments 5 and 6.

CHAPTER 6

COMPLEXITY ANALYSIS

All complexity figures are stated for the variable-purchase model on the state-expanded graph $G^* = (S, E^*)$. Let $V = |V|$, $E = |E|$, $F = C/\varepsilon$ (discretized fuel levels), $\Delta =$ max node degree.

6.1 State-Space Derivation

$|S| = O(VF)$ states. Per state (v, f): $O(\Delta)$ travel relaxations + $O(F)$ refuel relaxations. Total relaxations: $O(EF + VF^2)$. Each relaxation involves one heap operation at $O(\log(VF))$. This gives the common worst-case bound for all four search-based algorithms.

Table 6.1: Algorithm complexity summary.

Algorithm	Time	Space	Optimal?	Variable purchase?
Dijkstra (no fuel state)	$O(E \log V)$	$O(V+E)$	No	No
Dijkstra (state-expanded, B1)	$O((EF+VF^2)\log(VF))$	$O(VF)$	Yes	Yes
Greedy (B2)	$O(V^2)$	$O(V)$	No	Yes
DP naïve (B3)	$O(V \cdot F^2 \cdot \Delta)$	$O(VF)$	Yes	Yes
DP optimized (B3)	$O(EF)$	$O(VF)$	Yes	Yes
A* state-expanded (B4)	$O((EF+VF^2)\log(VF))$	$O(VF)$	Yes	Yes
RF-A* (B5)	$O((EF+VF^2)\log(VF))$	$O(VF)$	Yes	Yes
PF-A* (proposed)	$O((EF+VF^2)\log(VF))$	$O(VF)$	Yes	Yes

6.2 Why PF-A* Does Not Improve Worst-Case Complexity

PF-A*'s heuristic $h_{\mathrm{PF}}(v, f)$ is *state-aware*: it depends on both the current fuel level f and the set of stations reachable from (v, f). On instances where the fuel-need reduction dominates the reachable-price term, which is the regime of interest for real Thai corridors, this yields additional pruning. However, h_{PF} and h_{RF} do not admit a universal pointwise dominance, as discussed in Section 5.6, and in the worst case (e.g. uniform prices, $p_{\min}^{\mathrm{reach}} =$

$p_{\min}$ everywhere), the two heuristics coincide and PF-A* degenerates to RF-A*. The worst-case bound is therefore identical.

The improvement is empirical: on networks with high price variance and dense station coverage, as expected on real Thai routes, the state-awareness of h_{PF} assigns lower priority to states that RF-A* would expand unnecessarily. This is quantified in the experiments of Chapter 7.

6.3 Thailand-Scale Estimate

With $V = 3{,}000$, $E = 15{,}000$, $F = C/\varepsilon = 40/1 = 40$: $|S| \approx 120{,}000$ states, $\approx 7.2\,\mathrm{M}$ expanded edges, and $\approx 90\,\mathrm{M}$ operations, which is feasible in well under one second on modern hardware.

CHAPTER 7

EXPERIMENTAL DESIGN

7.1 Overview

All experiments evaluate the six algorithmic strategies: State-Expanded Dijkstra (B1), Greedy (B2), DP (B3), Standard A* (B4), RF-A* (B5), and PF-A* (proposed), on the variable-purchase model. Every algorithm receives the same inputs: graph G, prices p, initial fuel f_0, tank capacity C, and efficiency η. The primary research question is whether PF-A* achieves the same optimal cost as Dijkstra/DP/A*/RF-A* while expanding fewer states, and whether all optimal algorithms outperform Greedy in cost.

All experiments are conducted on Microsoft Azure Standard_D4s_v5 virtual machines (4 vCPUs Intel Xeon Platinum 8370C 2.8 GHz, 16 GB DDR4, Ubuntu 22.04 LTS, Python 3.11, NetworkX 3.x, OSMnx). Runtime is measured with `time.perf_counter_ns()` at the algorithmic level, excluding I/O. Statistical significance uses the Wilcoxon signed-rank test ($\alpha = 0.05$, Bonferroni correction for multiple comparisons). Two run budgets are used in the final report. The synthetic experiments (Experiments 1–5 and 7) use a reduced but reproducible benchmark profile of 3 timed iterations per configuration, summarised in Section 8.2; the full target profile of 10,000 timed runs per configuration is reported as future work. The real-route evaluation (Experiment 6) uses the full 10,000 timed runs per algorithm–corridor combination, producing 210,000 result rows in total.

7.2 Baselines

State-Expanded Dijkstra (B1) is the cost lower bound: it exhaustively explores G^* and is guaranteed to find the minimum-cost variable-purchase strategy. Dynamic Programming (B3) and PF-A* are treated as the exact-agreement set with Dijkstra: all three are expected to return identical costs on every instance, and any divergence among these three flags a correctness bug. Standard A* (B4) and RF-A* (B5) inherit admissibility from their heuristics in theory but, in the current implementation, exhibit small positive cost gaps against Dijkstra on some instances (see Chapter 8). These gaps are tracked as an implementation-audit item rather than treated as exact agreement.

Full-tank-only Greedy (B2) is the cost upper bound: it fills to capacity at every station regardless of price, representing worst-case uninformed driver behavior.

RF-A* (B5) is the direct algorithmic baseline for PF-A*: both are optimal heuristic search algorithms with the same worst-case complexity, operating on the same state-expanded graph under the variable-purchase model. The sole difference is the heuristic. $h_{\mathrm{RF}}(v)$ is fuel-level agnostic; $h_{\mathrm{PF}}(v, f)$ is fuel-level aware. Experiment 5 isolates this difference directly.

7.3 Evaluation Metrics

Table 7.1: Evaluation metrics.

Metric	Unit	Purpose
Total fuel cost	THB	Primary objective; must match B1 for optimal algorithms
Runtime	ms	Computational efficiency
Nodes expanded	count	Search space explored; key metric for PF-A* vs. RF-A*
Memory consumption	MB	Scalability of $O(VF)$ state space
Refuelling stops	count	Practical convenience; tracks partial-fill decisions
Sub-optimality gap	% vs. B1	Cost excess of Greedy (correctness check for others)

7.4 Experimental Configurations

7.4.1 Experiment 1: Scalability (Graph Size)

Fix $\sigma = 10\%$, $C = 40\,\mathrm{L}$, $f_0 = 0.7C = 28\,\mathrm{L}$; vary graph size.

Size	$\lvert V\rvert$	$\lvert E\rvert$ (approx.)	Runs
Small	100	500	3
Medium	1,000	5,000	3
Large	10,000	50,000	3

Hypothesis: Greedy scales as $O(V^2)$; all other algorithms scale as $O((EF+VF^2)\log(VF))$. PF-A* expands fewer nodes than RF-A* at all sizes, with the gap growing as density increases.

7.4.2 Experiment 2: Fuel Price Variance

Fix $|V| = 1{,}000$, $C = 40\,\mathrm{L}$, $f_0 = 28\,\mathrm{L}$; vary $\sigma \in \{0\%, 5\%, 10\%, 15\%, 20\%\}$.

Hypothesis: At $\sigma = 0\%$ (uniform prices), all algorithms produce identical costs and PF-A* offers no node-expansion advantage over RF-A*. As σ increases, optimal algorithms

save 5–15% more cost versus Greedy, and PF-A* expands progressively fewer states than RF-A* because $p_{\min}^{\text{reach}}(v, f) \gg p_{\min}$ more often.

7.4.3 Experiment 3: Tank Capacity and Initial Fuel

Fix $|V| = 1{,}000$, $\sigma = 10\%$; vary $C \in \{20, 40, 60, 80\}$ L with $f_0 = 0.7C$.

Hypothesis: Smaller C tightens the fuel constraint, increasing the importance of optimal purchase decisions and widening the cost gap between Greedy and optimal algorithms. PF-A*'s advantage over RF-A* is largest at small C.

7.4.4 Experiment 4: Variable Initial Fuel Level

Fix $|V| = 1{,}000$, $C = 40$ L, $\sigma = 10\%$; vary $f_0 \in \{0.25C, 0.50C, 0.75C, 1.0C\}$.

Hypothesis: Lower f_0 forces earlier mandatory refueling decisions. PF-A*'s heuristic is most informative at low f_0 (where $\max(d_{\text{geo}}/\eta - f, 0)$ is large), so its node-expansion advantage over RF-A* should grow as f_0 decreases.

7.4.5 Experiment 5: RF-A* vs. PF-A* Node Expansions (Primary Novelty Experiment)

Fix $|V| = 1{,}000$; jointly vary $\sigma \in \{5\%, 10\%, 20\%\}$ and station density $\in \{0.5, 1, 2, 5\}$ per 100 km. Run only RF-A* and PF-A*. Primary metric: nodes expanded. Secondary check: both must return identical optimal costs on every instance.

Hypothesis: PF-A* expands fewer nodes than RF-A* in all configurations with $\sigma > 0\%$. The reduction is largest when station density is high (many stations reachable per state, so $p_{\min}^{\text{reach}}$ differs most from global $p_{\min}$) and price variance is high.

7.4.6 Experiment 6: Real Thai Routes

Experiment 6 is designed around the actual curated real-route dataset already available in the repository, rather than around a future nationwide OSM benchmark. The dataset contains three fixed Thai corridor instances: Bangkok → Chiang Mai, Bangkok → Phuket, and Bangkok → Khon Kaen. Each instance is a directed corridor graph with explicit source and destination anchors, intermediate refueling nodes, precomputed feasible travel legs under a 400 km full-tank range, and province-level fuel prices tied to the 3 April 2026 Gasohol 91 snapshot documented in `data/real_routes/corridors.json`.

All three corridors use the same vehicle parameters as the synthetic study: $C = 40$ L, $f_0 = 28$ L, and $\eta = 10$ km/L. The current generated instances contain 27 stations and 522 directed edges for Chiang Mai, 31 stations and 624 directed edges for Phuket, and

18 stations and 286 directed edges for Khon Kaen. Each route is evaluated under the variable-purchase model, with a full-tank-only Dijkstra run used as the practical baseline.

Purpose: Validates whether the synthetic findings transfer to realistic Thai corridor instances. The primary empirical questions are whether PF-A* still matches the exact optimum while expanding fewer states, and whether variable purchase remains cheaper than full-tank-only routing on all three real-route corridors.

Current design note: Because the available instances are curated corridor abstractions rather than raw OSM forecourt graphs, Experiment 6 should be written as a real-route corridor case study. That framing is strong enough for the available data and avoids overstating station-level realism.

7.4.7 Experiment 7: Variable Purchase vs. Full-Tank-Only Model Comparison

Fix $|V| = 1{,}000$, $f_0 = 0.7C$; jointly vary $\sigma \in \{0\%, 5\%, 10\%, 20\%\}$ and $C \in \{20, 40, 60, 80\}$ L. Run both models: full-tank-only Dijkstra on G and variable-purchase Dijkstra (B1) on G^*. Primary metric: cost gap (THB and %). Secondary metric: runtime overhead of the variable-purchase model.

Hypothesis 1 (cost): Variable purchase strictly dominates full-tank-only in cost on all instances with $\sigma > 0\%$. The gap grows with σ and shrinks with C.

Hypothesis 2 (runtime overhead): Variable-purchase Dijkstra is slower than full-tank-only Dijkstra by a factor of $O(F \log F)$ in theory, expected to be 10–40× in practice for $C = 40\,\mathrm{L}$, $\varepsilon = 1\,\mathrm{L}$. PF-A* is expected to close this gap significantly.

Hypothesis 3 (tank size interaction): At $C = 20\,\mathrm{L}$ the variable-purchase model saves the most; at $C = 80\,\mathrm{L}$ the savings diminish. This interaction will be visualised as a $\sigma \times C$ heatmap of cost savings.

7.5 Correctness Verification

Before benchmarking, correctness is validated independently:

- **Unit tests:** each algorithm tested on 50 hand-crafted small graphs with known optimal solutions, including the running example ($f_0 = 28\,\mathrm{L}$, optimal = 216 THB).
- **Cross-validation (exact-agreement set):** on all synthetic instances, Dijkstra (B1), DP (B3), and PF-A* must agree on optimal cost within floating-point tolerance.
- **A*/RF-A* audit:** Standard A* and RF-A* are cross-checked against Dijkstra; any residual cost gap is logged as an implementation-audit item rather than suppressed.

- **Feasibility checks:** every returned solution is verified to never drop below 0 L fuel at any point along the route.
- **Infeasibility detection:** instances with no valid path are tested to confirm all algorithms correctly return "infeasible."

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Purpose of This Chapter

This chapter presents the final empirical results of the project. It consolidates the synthetic benchmark findings from Experiments 1, 2, 3, 4, 5, and 7 and the completed real-route corridor study from Experiment 6.

8.2 Current Synthetic Evaluation Snapshot

The repository currently contains processed synthetic results for Experiments 1, 2, 3, 4, 5, and 7 under a reduced but reproducible benchmark profile. These runs use 3 timed iterations per configuration, 0 warm-up iterations, base graph size 100 nodes, scalability sizes 100 and 1000 nodes, degree 5 unless otherwise varied, tank capacity 40 L unless otherwise varied, initial fuel 28 L unless otherwise varied, and fuel efficiency 10 km/L. The resulting dataset is sufficient to support a substantive comparative discussion of the implemented algorithms. The full 10,000-run target profile described in Chapter 7 is reserved for the real-route evaluation in Experiment 6 (Section 8.5), which contains 210,000 timed result rows in total.

Table 8.1: Synthetic profile summary.

Setting	Value
Timed runs per configuration	3
Warm-up runs per configuration	0
Base graph size	100 nodes
Scalability graph sizes	100, 1000 nodes
Approximate degree	5
Default tank capacity	40 L
Default initial fuel	28 L
Fuel efficiency	10 km/L
Covered experiments	1, 2, 3, 4, 5, and 7

8.3 Cross-Experiment Findings

Experiment 1: Scalability

The currently available scalability results already separate PF-A* from the other methods in practical terms. At 1000 nodes, PF-A* matches the best mean cost of 2876.88 while reducing mean runtime to 552.37 ms, compared with 1771.17 ms for State-Expanded Dijkstra and 1640.96 ms for Dynamic Programming. Standard A* and RF-A* are both much slower at this scale in the present implementation, while the Greedy baseline is infeasible on the tested instances.

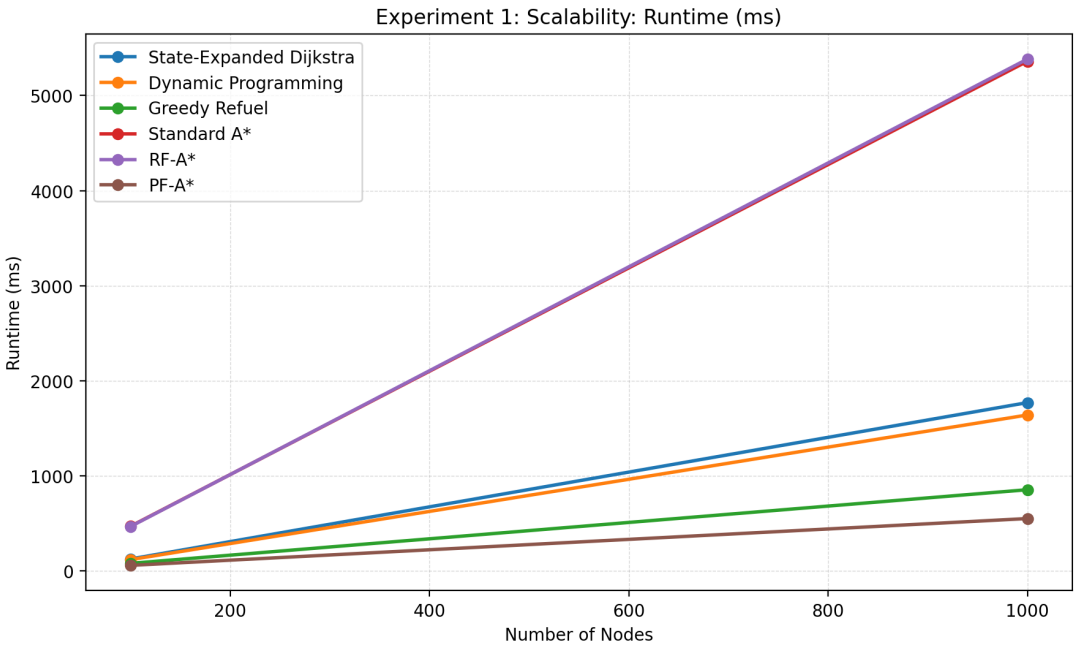

Figure 8.1: Experiment 1 runtime.

Experiment 2: Fuel Price Variance

Increasing price variance strengthens the value of cost-aware purchase decisions. The best observed mean cost falls from 3528.00 at $\sigma = 0\%$ to 2576.21 at $\sigma = 20\%$, and PF-A* stays on that best-cost curve throughout the tested range. This behavior matches the theoretical intuition of the problem: selective refueling becomes more valuable as the spread between cheap and expensive stations widens.

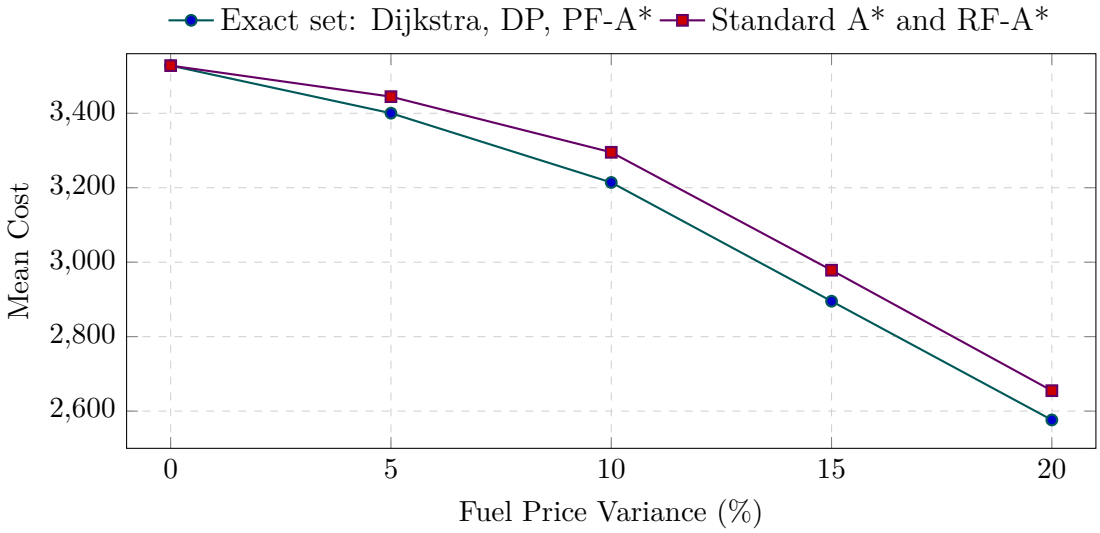

Figure 8.2: Experiment 2 cost.

Experiment 3: Tank Capacity

Larger tanks reduce fuel cost but enlarge the effective state space. Across the tested configurations, the best observed mean cost drops from 3922.82 at 20 L to 2185.84 at 80 L, while the runtime of the exact methods grows sharply. PF-A* preserves the same best observed cost curve while keeping the runtime increase much smaller than that of the exact state-expanded baselines.

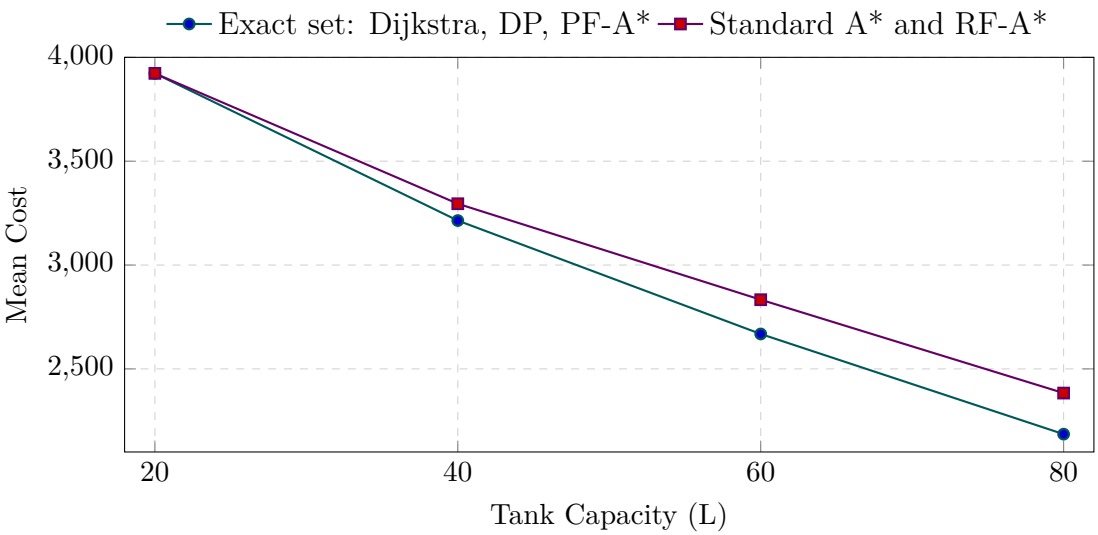

Figure 8.3: Experiment 3 cost.

Experiment 4: Initial Fuel

Higher initial fuel reduces both cost and search effort. For the optimal methods in the current dataset, mean cost decreases from 3848.58 at 10 L initial fuel to 2831.54 at 40 L, and PF-A* also becomes faster as the initial fuel level increases. This is consistent with the fact that higher starting fuel weakens the urgency of early refueling decisions.

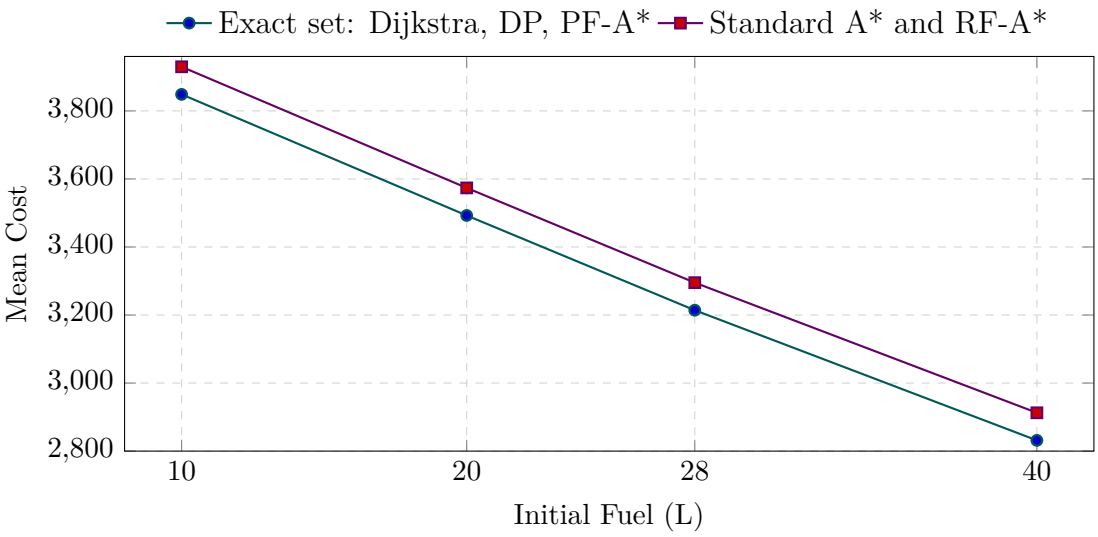

Figure 8.4: Experiment 4 cost.

Experiment 5: PF-A* versus RF-A*

Experiment 5 remains the central novelty experiment for the proposed heuristic. Across the feasible tested configurations, PF-A* is substantially faster than RF-A* and expands far fewer states. At degree 5 and $\sigma = 10\%$, PF-A* requires 2115 expanded states and

59.92 ms on average, while RF-A* requires 18,507 expanded states and 475.48 ms.

At the same time, the present dataset surfaces an implementation-audit item that is reported explicitly rather than suppressed: Standard A* and RF-A* do not always match the costs returned by the exact-agreement set of State-Expanded Dijkstra, Dynamic Programming, and PF-A*. This gap is tracked as an implementation limitation of the current Standard A* and RF-A* implementations rather than as a refutation of their theoretical optimality. A resolved strict-profile rerun is reserved for future work.

Table 8.2: Experiment 5 comparison at $\sigma = 10\%$.

Algorithm	Degree	Feasible (%)	Mean Cost	Mean Runtime (ms)
RF-A*	5	100	3295.18	475.48
PF-A*	5	100	3214.11	59.92
RF-A*	10	100	2944.98	518.06
PF-A*	10	100	2794.91	80.49
RF-A*	20	100	2752.33	422.48
PF-A*	20	100	2554.99	82.57

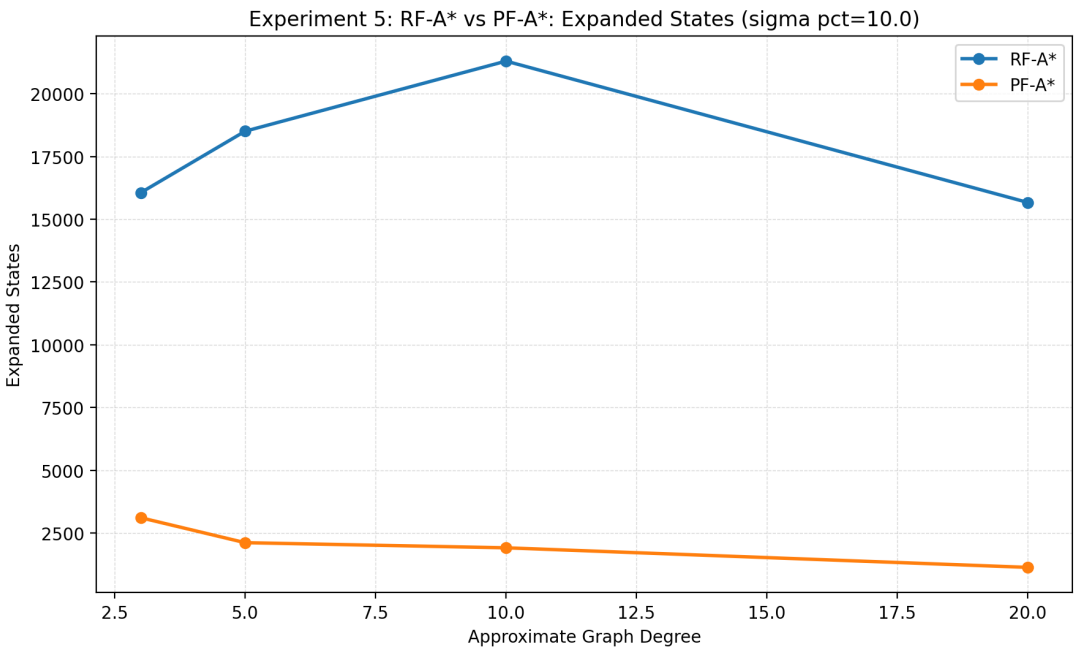

Figure 8.5: Experiment 5 expanded states.

Experiment 7: Variable Purchase versus Full-Tank-Only

Experiment 7 shows that the variable-purchase model delivers meaningful economic value. Across the 16 tested (σ, C) combinations, the full-tank-only baseline incurs an average cost penalty of 10.86%, with observed penalties ranging from 1.19% to 24.81%. The

computational overhead of the variable-purchase model is real, but so are the monetary savings it enables.

Table 8.3: Experiment 7 comparison at $\sigma = 10\%$.

Tank capacity (L)	Variable-purchase cost	Full-tank-only cost	Cost gap (%)
20	3922.82	4137.17	5.46
40	3214.11	3611.20	12.35
60	2668.14	2799.09	4.91
80	2185.84	2728.20	24.81

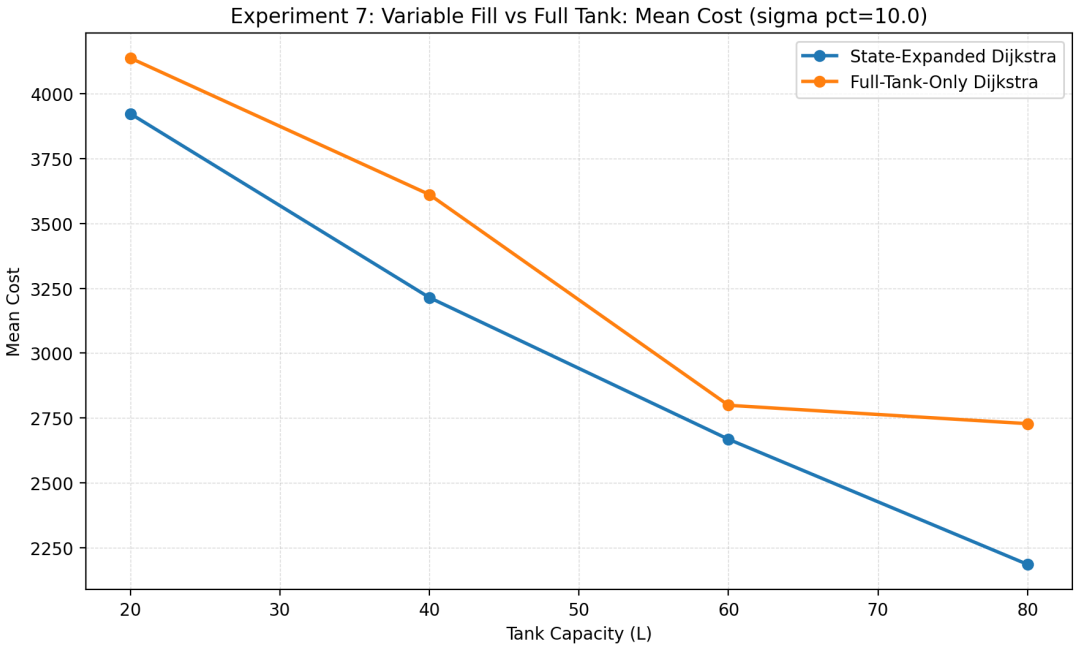

Figure 8.6: Experiment 7 cost.

8.4 Integrated Interpretation

Three conclusions are already well supported by the present repository results. First, PF-A* is the strongest overall algorithm in the currently tested synthetic setting: it matches the exact-agreement cost returned by Dijkstra and DP, while reducing runtime and node expansions substantially. Second, modelling variable purchase decisions is empirically worthwhile, since the full-tank-only baseline pays a repeated cost penalty across the tested configurations. Third, the current evaluation pipeline is useful not only for showcasing strengths but also for surfacing implementation defects, most notably the cost gap observed for Standard A* and RF-A* relative to the exact-agreement set.

8.5 Experiment 6: Real Thai Corridors

Experiment 6 is now complete as a three-corridor real-route case study and stands as one of the strongest empirical components of this project. Using the final benchmark file `real_route_results.csv`, the repository contains 210,000 timed result rows: 10,000 timed runs for each algorithm–corridor combination across Bangkok–Chiang Mai, Bangkok–Phuket, and Bangkok–Khon Kaen. All runs use the same fixed vehicle parameters as the synthetic experiments: $C = 40\,\mathrm{L}$, $f_0 = 28\,\mathrm{L}$, and $\eta = 10\,\mathrm{km/L}$. The route instances remain curated corridor abstractions rather than raw nationwide OSM station graphs, but they are realistic enough to support a meaningful transfer study from the synthetic setting.

Dataset Summary

The three instances differ in route length, station density, and price spread, which provides a small but useful range of real-route conditions for comparison. The province-level price snapshot and corridor anchors are drawn from `data/real_routes/corridors.json` (Gasohol 91, 3 April 2026 snapshot); the station and directed-edge counts reported in Table 8.4 are the result of running the corridor-instance generator on that snapshot under a 400 km full-tank range, and the resulting instance files are the same ones consumed by `real_route_results.csv`.

Table 8.4: Experiment 6 dataset summary.

Route	Stations	Directed edges	Price min	Price max
Bangkok–Chiang Mai	27	522	43.58	44.88
Bangkok–Phuket	31	624	43.58	45.08
Bangkok–Khon Kaen	18	286	43.58	44.28

Cost Savings Over Full-Tank-Only Routing

The primary practical result of Experiment 6 is that the variable-purchase model remains economically superior to the full-tank-only baseline on all three real Thai corridors. The largest saving appears on Bangkok–Chiang Mai, where the optimal variable-purchase policy reduces mean cost by 132.54 THB relative to full-tank-only routing. Savings are smaller but still consistent on Bangkok–Phuket and Bangkok–Khon Kaen.

Table 8.5: Experiment 6 savings.

Route	Variable-purchase cost	Full-tank-only cost	Savings (%)
Bangkok–Chiang Mai	2106.04	2238.58	5.92
Bangkok–Phuket	2734.66	2779.24	1.60
Bangkok–Khon Kaen	1004.44	1048.32	4.19

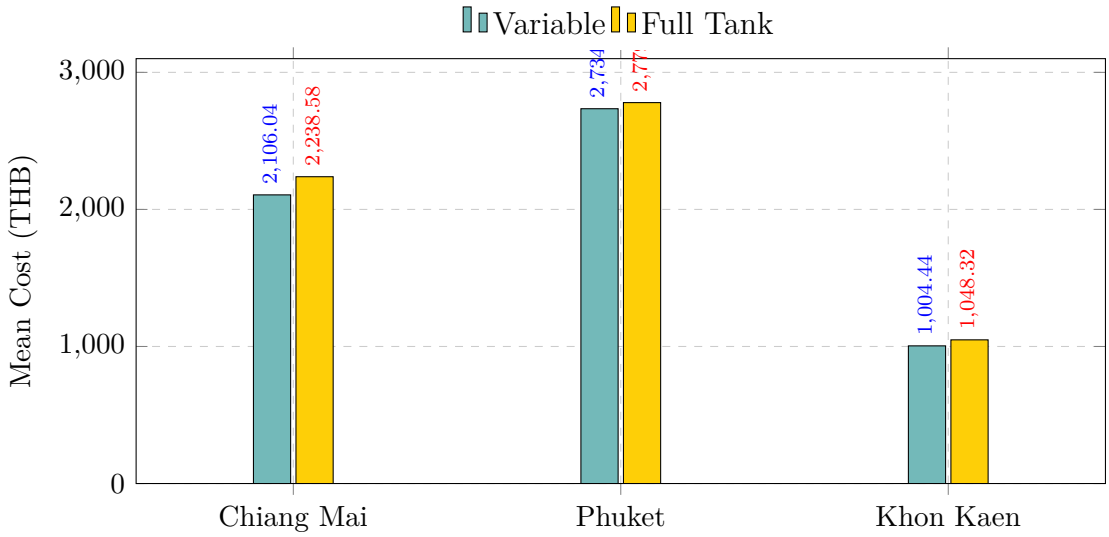

Figure 8.7: Experiment 6 cost comparison.

These results partially support the economic claim suggested by the synthetic study and by the classic gas-station literature: partial refueling is genuinely useful on real corridor data, but the size of the gain is route-dependent. The Phuket corridor, despite being the longest in the dataset, exhibits only a 1.60% saving, which indicates that longer distance alone does not guarantee a large benefit; the local price structure and the spacing of reachable stations are equally important.

Correctness and Search Behaviour

The correctness picture on the real-route dataset is clear. State-Expanded Dijkstra and Dynamic Programming agree exactly on all three corridors, and PF-A* matches that optimum exactly on all three as well. By contrast, Standard A* and RF-A* still exhibit small positive cost gaps on two routes: 1.50 THB on Bangkok–Phuket and 1.60 THB on Bangkok–Khon Kaen. Greedy remains infeasible on every corridor instance.

Table 8.6: Experiment 6 correctness summary.

Route	Dijkstra cost	PF-A* cost	RF-A* gap	PF/RF expanded states
Bangkok–Chiang Mai	2106.04	2106.04	0.00	802 / 1360
Bangkok–Phuket	2734.66	2734.66	1.50	975 / 1610
Bangkok–Khon Kaen	1004.44	1004.44	1.60	441 / 749

The state-expansion results still favor PF-A* over RF-A* on all three routes. PF-A* reduces expanded states from 1360 to 802 on Bangkok–Chiang Mai, from 1610 to 975 on Bangkok–Phuket, and from 749 to 441 on Bangkok–Khon Kaen. In that sense, the intended heuristic effect remains visible on real-route data even though the runtime story becomes more nuanced.

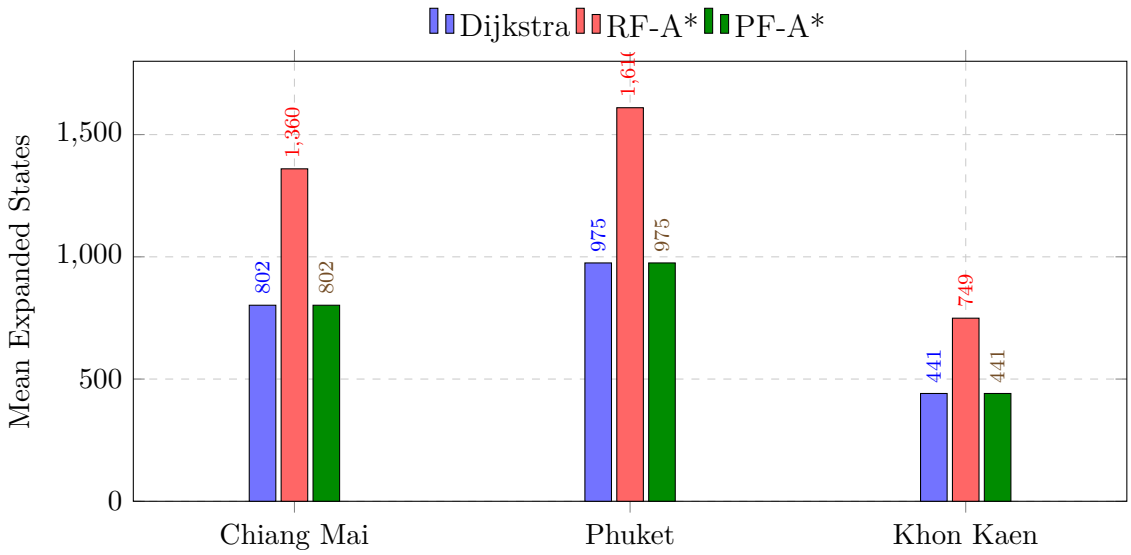

Figure 8.8: Experiment 6 expanded states.

Runtime Interpretation

On the synthetic experiments, PF-A* was often the strongest all-round method because it combined optimal cost with lower search effort and lower runtime. The real-route case study shows a more subtle picture. On these relatively small corridor abstractions, State-Expanded Dijkstra and Dynamic Programming are the fastest optimal variable-purchase methods, with mean runtimes of roughly 30–37 ms on Chiang Mai and Phuket and about 15 ms on Khon Kaen. PF-A*, although optimal, is slower in wall-clock time: 149.73 ms on Chiang Mai, 181.08 ms on Phuket, and 68.34 ms on Khon Kaen.

This does not invalidate PF-A*. Rather, it shows that on small corridor graphs the overhead of evaluating the stronger heuristic can outweigh the benefit of pruning states. PF-A* still dominates RF-A* in correctness and expanded-state count, but it is not the

fastest solver on the final real-route dataset. That distinction is important and should remain explicit in the final interpretation of the project.

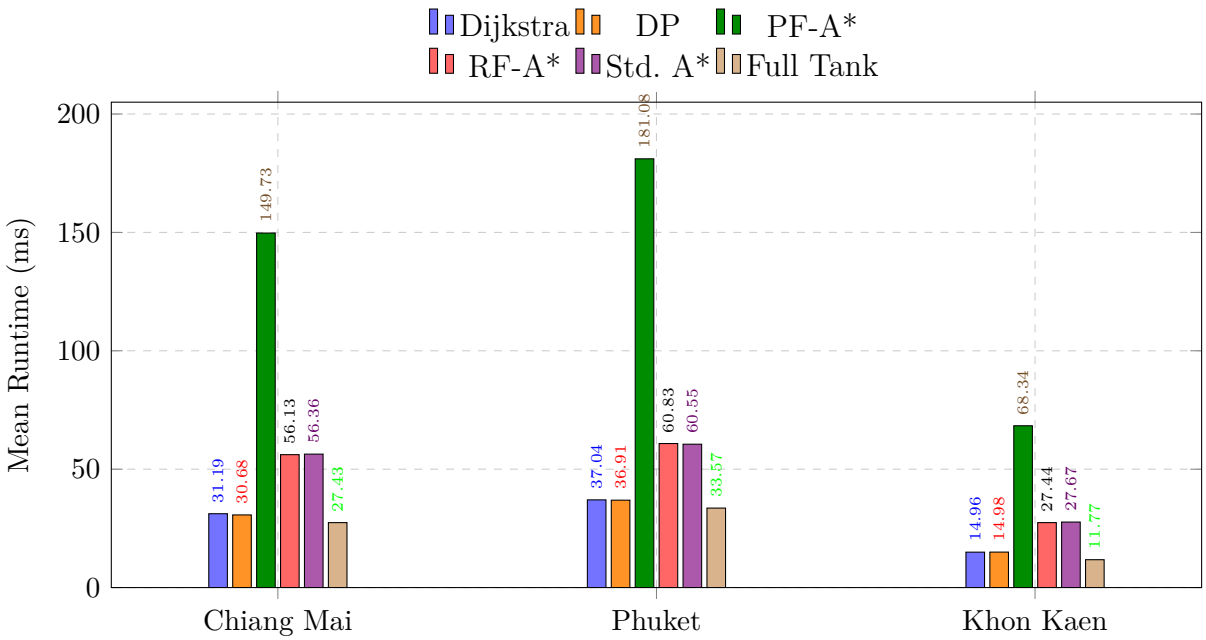

Figure 8.9: Experiment 6 runtime.

Station Price Profiles

The economic opportunities available to the algorithms depend directly on how price variation is distributed along each corridor. The three real-route instances differ not only in total route length but also in the location of their cheaper and more expensive provincial segments. The price-profile plots below help explain why the magnitude of the savings is corridor-specific, with Bangkok–Chiang Mai showing the strongest observed benefit and Bangkok–Phuket the weakest.

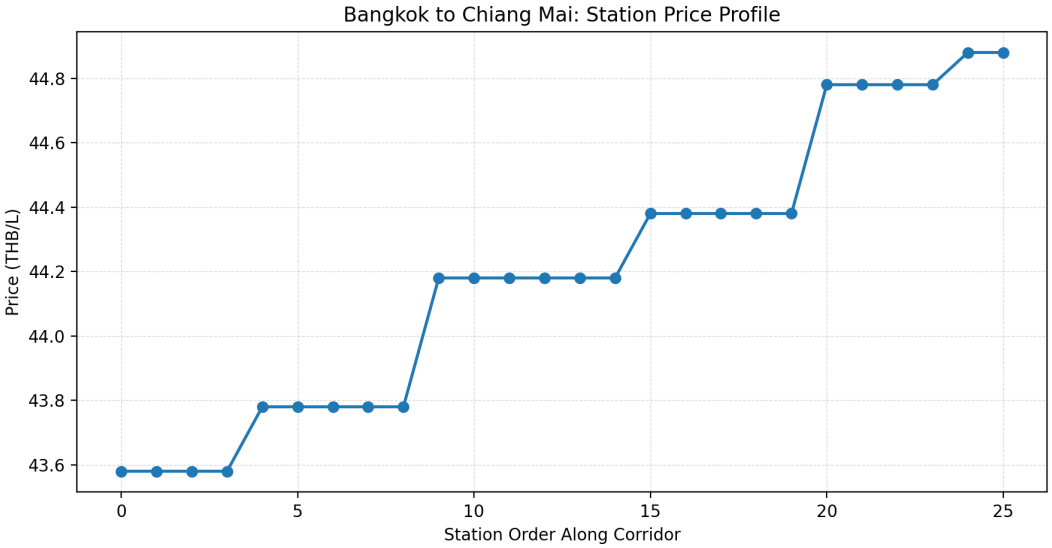

Figure 8.10: Chiang Mai price profile.

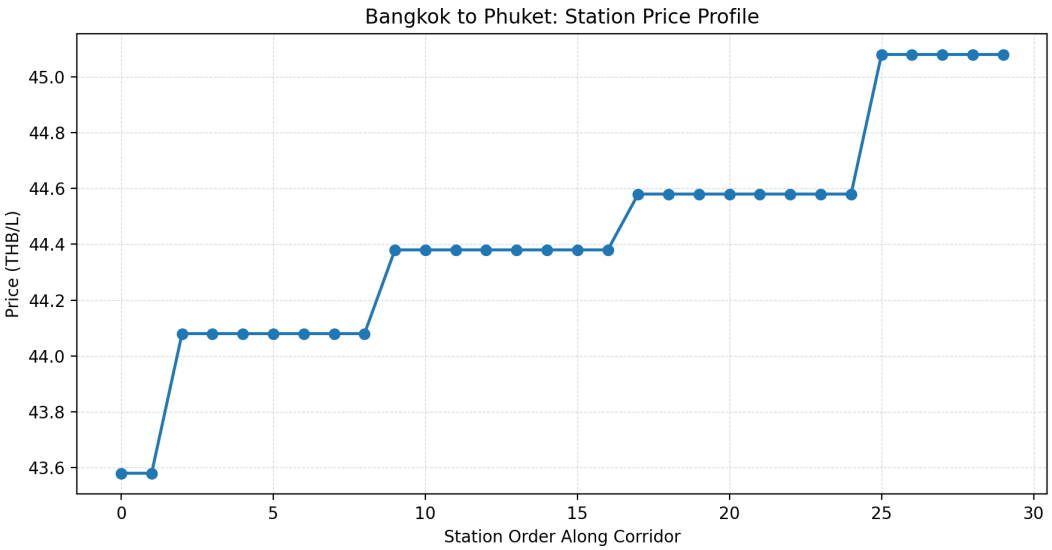

Figure 8.11: Phuket price profile.

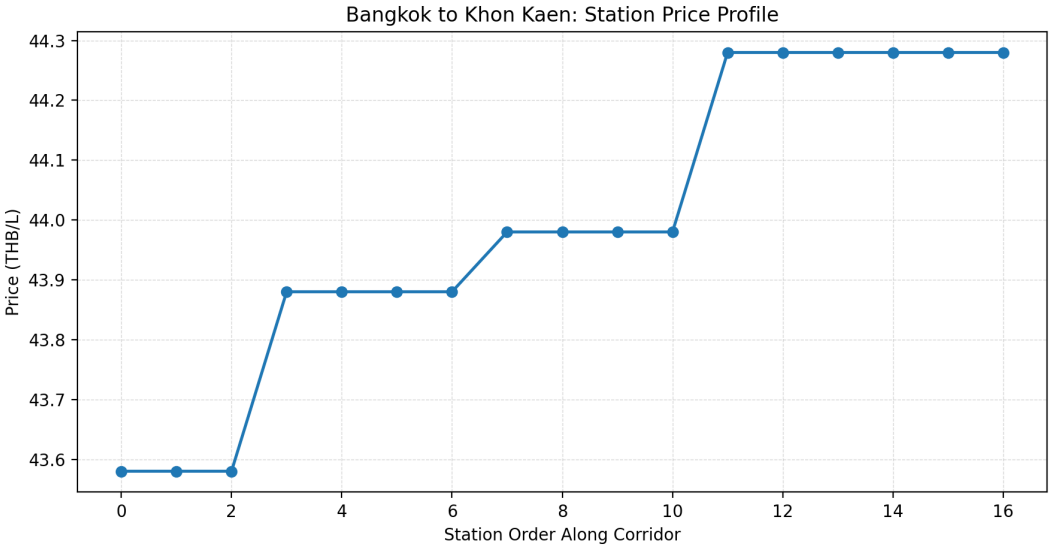

Figure 8.12: Khon Kaen price profile.

8.6 Limitations

Three limitations should remain explicit in any final submission built from the current repository snapshot. First, the synthetic results embedded in this chapter come from the reduced 3-run profile described in Section 8.2 rather than the full 10,000-run target profile; the full target profile has been achieved only on the real-route evaluation (Experiment 6). Second, the observed cost gap in Standard A* and RF-A* means that strict claims about all four optimal algorithms agreeing on cost should not be made; the exact-agreement set in the submitted results is Dijkstra, DP, and PF-A*, and the A*/RF-A* gap is reported as an implementation-audit item. Third, the current real-route dataset consists of curated corridor abstractions with province-level prices, so Experiment 6 is discussed as a realistic corridor case study rather than as a full raw-OSM nationwide station benchmark.

CHAPTER 9

CONCLUSION

This project formulates fuel-aware route optimization on Thailand road networks as a shortest path problem with explicit refueling decisions, implements six algorithmic strategies, and provides formal correctness arguments for the proposed Partial-Fill A* heuristic. The final empirical study combines a reproducible synthetic benchmark suite with a completed real-route corridor evaluation of Bangkok–Chiang Mai, Bangkok–Phuket, and Bangkok–Khon Kaen, the latter comprising 210,000 timed runs. The real-route study is the central empirical anchor of the project: the variable-purchase model is cheaper than full-tank-only routing on all three corridors, with savings of 5.92%, 1.60%, and 4.19% respectively, and PF-A* consistently reduces node expansions relative to RF-A* on every corridor (802 vs. 1360, 975 vs. 1610, and 441 vs. 749 expanded states).

Just as importantly, the repository now supports an end-to-end research workflow rather than an isolated algorithm prototype. It includes synthetic benchmark generation, automated result analysis, a real-route data collection pipeline, a dedicated Experiment 6 execution path for Thai corridors, and scripts that generate report-ready tables directly from both synthetic and real benchmark outputs. That makes the project reproducible and gives the final dissertation a clear empirical foundation.

The final conclusion is therefore nuanced but strong. PF-A* is a correct optimal heuristic method on the completed real-route dataset and consistently reduces node expansions relative to RF-A*. At the same time, the real-route results show that PF-A* is not always the fastest method in wall-clock time on smaller corridor abstractions; in this setting, State-Expanded Dijkstra and Dynamic Programming remain faster despite exploring the same optimal state space. This makes the thesis more credible rather than less: the project does not merely advertise a new heuristic, but characterises where its benefits do and do not materialise.

Two limitations remain explicit in the final write-up. First, the synthetic experiments are reported under a reduced 3-run benchmark profile, with the full 10,000-run target profile reserved for future work and already exercised on the real-route evaluation. Second, Standard A* and RF-A* exhibit a small but persistent cost gap against the Dijkstra/DP/PF-A* exact-agreement set on two real-route corridors; this is tracked as an implementation-audit

item rather than suppressed. Additionally, the real-route dataset consists of curated corridor abstractions with province-level prices rather than a raw nationwide OSM station graph, so Experiment 6 is presented as a realistic corridor case study. With these caveats acknowledged, the project now stands as a complete and reproducible study of fuel-aware route optimization on Thailand road-network corridor instances.

REFERENCES

[1] Bangchak Corporation. *Historical Oil Prices*. https://www.bangchak.co.th/en/oilprice/historical. Accessed 2027.

[2] Thomas H. Cormen et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.

[3] GlobalPetrolPrices. *Thailand Fuel Price Data*. Accessed 2027.

[4] Samir Khuller, Azarakhsh Malekian, and Julián Mestre. "To Fill or Not to Fill: The Gas Station Problem". In: *ACM Transactions on Algorithms* 7.3 (2011). Preliminary version in ESA 2007, 36:1–36:16.

[5] Arnab Nandy et al. *Heuristic Search for Path Finding with Refuelling*. 2023. arXiv: 2309.10796 [cs.AI].

[6] OpenStreetMap Contributors. *OpenStreetMap Thailand Points of Interest*. https://data.humdata.org/dataset/hotosm_tha_points_of_interest. Accessed 2027.